

Nama : 1. Andre Hadiman Rotua Parhusip (122450108)
2. Nabyla Sharfina (123450008)
3. Arini Puteri Elandra (123450069)
4. Muhammad Fadil Alfaizi (123450115)

Kelompok : 14

Dosen Pembimbing : Yuliana M.Sc

MISI 3
IMPLEMENTASI PRODUKSI
FAKULTAS TEKNOLOGI INDUSTRI

1. Step 1: Production Deployment

Tujuan: Men-deploy database dan ETL ke Azure VM production

Aktivitas:

1.1. Environment Setup

- Provision Azure VM dengan SQL Server
- Configure firewall rules
- Setup SQL Server Service
- Configure backup locations

1.2. Database Deployment

```
-- Deploy using SSMS or sqlcmd
:r 01_Create_Database.sql
:r 01_Create_Database.sql
:r 02_Create_Dimensions.sql
:r 03_Create_Facts.sql
:r 04_Create_Indexes.sql
:r 05_Create_Partitions.sql
:r 06_Create_Staging.sql
:r 07_ETL_Procedures.sql
:r 08_Data_Quality_Checks.sql
```

1.3. Initial Data Load

- Execute full ETL untuk historical data
- Verify data integrity
- Document load statistics

1.4. Schedule ETL Jobs

```
-- Create SQL Server Agent Job
```

```

USE msdb;
GO

EXEC sp_add_job
    @job_name = N'ETL_Daily_Load',
    @enabled = 1,
    @description = N'Daily ETL load for Data Mart';
GO

EXEC sp_add_jobstep
    @job_name = N'ETL_Daily_Load',
    @step_name = N'Execute Master ETL',
    @subsystem = N'TSQL',
    @command = N'EXEC dbo.usp_Master_ETL;',
    @database_name = N'DM_FTI_DW',
    @retry_attempts = 3,
    @retry_interval = 5;
GO

EXEC sp_add_schedule
    @schedule_name = N'Daily at 2 AM',
    @freq_type = 4,      -- Daily
    @freq_interval = 1,  -- Every day
    @active_start_time = 020000; -- 02:00 AM
GO

EXEC sp_attach_schedule
    @job_name = N'ETL_Daily_Load',
    @schedule_name = N'Daily at 2 AM';
GO

EXEC sp_add_jobserver
    @job_name = N'ETL_Daily_Load';
GO

```

2. Step 2: Dashboard Development

Tujuan: Membangun interactive dashboard untuk end-users

Aktivitas:

2.1. Create Analytical Views

```

-- View: Prestasi Mahasiswa Summary
CREATE VIEW dbo.vw_Prestasi_Mahasiswa AS

```

```

SELECT
    dm.nim,
    dm.nama_mahasiswa,
    dp.nama_prodi,
    w.tahun,
    COUNT(fp.sk_prestasi) AS total_prestasi,
    COUNT(DISTINCT fp.sk_prestasi) AS jumlah_event,
    MIN(w.tahun) AS tahun_prestasi_pertama,
    MAX(w.tahun) AS tahun_prestasi_terbaru
FROM Fact_Prestasi fp
JOIN Dim_Mahasiswa dm ON fp.sk_mahasiswa = dm.sk_mahasiswa
JOIN Dim_ProgramStudi dp ON fp.sk_prodi = dp.sk_prodi
JOIN Dim_Waktu w ON fp.sk_waktu = w.sk_waktu
WHERE dm.IsCurrent = 1
GROUP BY dm.nim, dm.nama_mahasiswa, dp.nama_prodi, w.tahun;
GO

```

-- View: Program Studi Analytics

```

CREATE VIEW dbo.vw_Program_Analytics AS
SELECT
    dp.nama_prodi,
    w.tahun,
    COUNT(DISTINCT fp.sk_mahasiswa) AS total_mahasiswa_berprestasi,
    COUNT(fp.sk_prestasi) AS total_prestasi,
    COUNT(DISTINCT fa.sk_anggaran) AS jumlah_anggaran,
    SUM(fa.total_anggaran) AS total_anggaran,
    AVG(fak.rata_rata_ipk) AS rata_rata_ipk,
    AVG(fd.rasio_dosen_mahasiswa) AS rasio_dosen_mahasiswa
FROM Dim_ProgramStudi dp
LEFT JOIN Fact_Prestasi fp ON fp.sk_prodi = dp.sk_prodi
LEFT JOIN Fact_Anggaran fa ON fa.sk_prodi = dp.sk_prodi
LEFT JOIN Fact_Akademik fak ON fak.sk_prodi = dp.sk_prodi
LEFT JOIN Fact_Dosen fd ON fd.sk_prodi = dp.sk_prodi
LEFT JOIN Dim_Waktu w ON w.sk_waktu = fp.sk_waktu
GROUP BY dp.nama_prodi, w.tahun;
GO

```

-- View: Anggaran & Revenue Analysis

```

CREATE VIEW dbo.vw_Anggaran_Summary AS
SELECT
    dp.nama_prodi,
    w.tahun,
    SUM(fa.total_anggaran) AS total_anggaran,
    AVG(fak.jumlah_mahasiswa_baru) AS mahasiswa_baru,
    SUM(fd.jumlah_dosen) AS total_dosen
FROM Fact_Anggaran fa

```

```

JOIN Dim_ProgramStudi dp ON dp.sk_prodi = fa.sk_prodi
JOIN Dim_Waktu w ON w.sk_waktu = fa.sk_waktu
LEFT JOIN Fact_Akademik fak ON fak.sk_prodi = dp.sk_prodi
LEFT JOIN Fact_Dosen fd ON fd.sk_prodi = dp.sk_prodi
GROUP BY dp.nama_prodi, w.tahun;
GO

-- View: Executive Dashboard Base View
CREATE VIEW dbo.vw_Executive_Summary AS
SELECT
    w.tahun,
    COUNT(DISTINCT dm.sk_mahasiswa) AS total_mahasiswa,
    COUNT(DISTINCT dp.sk_prodi) AS total_program_studi,
    SUM(fa.total_anggaran) AS total_anggaran,
    AVG(fak.rata_rata_ipk) AS ipk_rata_rata,
    SUM(fp.jumlah_prestasi) AS total_prestasi
FROM Dim_Waktu w
LEFT JOIN Fact_Akademik fak ON fak.sk_waktu = w.sk_waktu
LEFT JOIN Fact_Anggaran fa ON fa.sk_waktu = w.sk_waktu
LEFT JOIN Fact_Prestasi fp ON fp.sk_waktu = w.sk_waktu
LEFT JOIN Dim_Mahasiswa dm ON dm.sk_prodi = fak.sk_prodi
LEFT JOIN Dim_ProgramStudi dp ON dp.sk_prodi = fak.sk_prodi
GROUP BY w.tahun;
GO

```

2.2. Design Power BI Dashboards

Dashboard 1: Executive Summary

- KPI Cards:
 - Total Mahasiswa
 - Total Prestasi
 - Total Anggaran
 - Rata-rata IPK
 - Rasio Dosen-Mahasiswa
- Line Chart: Trend Anggaran 6 Tahun
- Bar Chart: Prestasi dalam Setiap Program Studi
- Matrix: Tren Mahasiswa, Prestasi, dan IPK Tahunan
- Donut Chart: Distribusi Mahasiswa berdasarkan Prodi
- Execute full ETL untuk historical data
- Verify data integrity
- Document load statistics

Data Source:

VW_Executive_Summary, VW_Program_Analytics

Dashboard 2: Academic Performance

- Stacked Bar: IPK per Prodi
- Line Chart: Tren IPK Tahun ke Tahun
- Bar Chart: Jumlah Prestasi Mahasiswa per Tahun
- Scatter: IPK vs Prestasi per Program Studi
- Slicer:
 - Tahun
- Verify data integrity
- Document load statistics

Data Source:

VW_Prestasi_Mahasiswa, VW_Program_Analytics

Dashboard 3: Financial Analysis

- Area Chart : Revenue/Anggaran Tren per Tahun
- Column Chart: Anggaran Per Program Studi
- Waterfall: Anggaran per Jenjang Pendidikan
- Table: Perbandingan Total Anggaran Setiap Program studi per Tahun
- Execute full ETL untuk historical data
- Verify data integrity
- Document load statistics

Data Source:

VW_Anggaran_Summary

2.3. Implement Interactivity

- Slicers:
 - Tahun
 - Program Studi
- Drill-through: From summary to detail
- Cross-filtering between visuals
- Bookmarks:
 - Overview
 - Academic Details
 - Financial Details
 - Performance Heatmap
- Row-level security

3. Step 3: Security Implementation

Tujuan: Membangun interactive dashboard untuk end-users

Aktivitas:

3.1. Create User Roles

```

-- Create Roles
CREATE ROLE db_executive;
CREATE ROLE db_analyst;
CREATE ROLE db_viewer;
CREATE ROLE db_etl_operator;
GO

-----

-- Executive Permissions (Full Read + ETL)
-----

GRANT SELECT ON SCHEMA::dbo TO db_executive;
GRANT EXECUTE ON dbo.usp_Master_ETL TO db_executive;
GO

-----

-- Analyst Permissions (DW + Staging)
-----

GRANT SELECT ON SCHEMA::dbo TO db_analyst;
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::stg TO
db_analyst;
GO

-----

-- Viewer Permissions (Read-only Views)
-----

GRANT SELECT ON dbo.vw_Prestasi_Mahasiswa TO db_viewer;
GRANT SELECT ON dbo.vw_Program_Analytics TO db_viewer;
GRANT SELECT ON dbo.vw_Executive_Summary TO db_viewer;
GO

-----

-- ETL Operator Permissions
-----

GRANT EXECUTE ON SCHEMA::dbo TO db_etl_operator;
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::stg TO
db_etl_operator;
GRANT INSERT ON SCHEMA::dbo TO db_etl_operator;
GO

```

3.2. Create Users and Assign Roles

```

-- Create SQL Logins
CREATE LOGIN executive_user WITH PASSWORD = 'StrongP@ssw0rd!';
CREATE LOGIN analyst_user WITH PASSWORD = 'StrongP@ssw0rd!';
CREATE LOGIN viewer_user WITH PASSWORD = 'StrongP@ssw0rd!';

```

```

CREATE LOGIN etl_service WITH PASSWORD = 'StrongP@ssw0rd!';
GO

-- Create Database Users
USE DM_FTI_DW;
GO

CREATE USER executive_user FOR LOGIN executive_user;
CREATE USER analyst_user FOR LOGIN analyst_user;
CREATE USER viewer_user FOR LOGIN viewer_user;
CREATE USER etl_service FOR LOGIN etl_service;
GO

-- Assign to Roles
ALTER ROLE db_executive ADD MEMBER executive_user;
ALTER ROLE db_analyst ADD MEMBER analyst_user;
ALTER ROLE db_viewer ADD MEMBER viewer_user;
ALTER ROLE db_etl_operator ADD MEMBER etl_service;
GO

```

3.3. Implement Data Masking

```

-- Masking Rules
ALTER TABLE dbo.Dim_Mahasiswa
ALTER COLUMN Email ADD MASKED WITH (FUNCTION = 'email()');

ALTER TABLE dbo.Dim_Mahasiswa
ALTER COLUMN Telp ADD MASKED WITH (FUNCTION =
'partial(0,"XXX-XXX-",4)');
GO

-- Grant UNMASK permission untuk specific roles
GRANT UNMASK TO db_executive;
GRANT UNMASK TO db_analyst;
GO

```

3.4. Implement Audit Trail

```

-- Create Audit Table
CREATE TABLE dbo.AuditLog (
    AuditID BIGINT IDENTITY(1,1) PRIMARY KEY,
    EventTime DATETIME2 DEFAULT SYSDATETIME(),
    UserName NVARCHAR(128) DEFAULT SUSER_SNAME(),
    EventType NVARCHAR(50),
    SchemaName NVARCHAR(128),

```

```

ObjectName NVARCHAR(128),
SQLStatement NVARCHAR(MAX),
RowsAffected INT,
IPAddress VARCHAR(50),
ApplicationName NVARCHAR(128) DEFAULT APP_NAME()
);
GO

-- Create Audit Trigger (example for Fact_Enrollment)
CREATE TRIGGER trg_Audit_Fact_Prestasi
ON dbo.Fact_Prestasi
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @EventType NVARCHAR(50);

    IF EXISTS (SELECT 1 FROM inserted) AND EXISTS (SELECT 1 FROM
deleted)
        SET @EventType = 'UPDATE';
    ELSE IF EXISTS (SELECT 1 FROM inserted)
        SET @EventType = 'INSERT';
    ELSE
        SET @EventType = 'DELETE';

    INSERT INTO dbo.AuditLog (EventType, SchemaName, ObjectName,
RowsAffected)
    VALUES (@EventType, 'dbo', 'Fact_Prestasi', @@ROWCOUNT);
END;
GO

-- Enable SQL Server Audit (Server-level)
CREATE SERVER AUDIT DataWarehouse_Audit
TO FILE
(
    FILEPATH = N'D:\Audit\',
    MAXSIZE = 100 MB,
    MAX_ROLLOVER_FILES = 10
)
WITH (ON_FAILURE = CONTINUE);
GO

ALTER SERVER AUDIT DataWarehouse_Audit WITH (STATE = ON);
GO

```

```
-- Create Database Audit Specification
CREATE DATABASE AUDIT SPECIFICATION DataWarehouse_DB_Audit
FOR SERVER AUDIT DataWarehouse_Audit
ADD (SELECT, INSERT, UPDATE, DELETE ON SCHEMA::dbo BY public);
GO

ALTER DATABASE AUDIT SPECIFICATION DataWarehouse_DB_Audit
WITH (STATE = ON);
GO
```

4. Step 4: Backup and Recovery Strategy

Tujuan: Menyiapkan backup untuk disaster recovery

Aktivitas:

```
-- Full Backup
BACKUP DATABASE DM_FTI_DW
TO DISK = N'D:\Backup\DM_FTI_DW_Full.bak'
WITH
    COMPRESSION,
    INIT,
    NAME = N'Full Database Backup',
    STATS = 10;
GO

-- Differential Backup
BACKUP DATABASE DM_FTI_DW
TO DISK = N'D:\Backup\DM_FTI_DW_Diff.bak'
WITH
    DIFFERENTIAL,
    COMPRESSION,
    INIT,
    NAME = N'Differential Database Backup',
    STATS = 10;
GO

-- Transaction Log Backup
BACKUP LOG DM_FTI_DW
TO DISK = N'D:\Backup\DM_FTI_DW_Log.trn';
WITH
    COMPRESSION,
    INIT,
    NAME = N'Transaction Log Backup',
    STATS = 10;
GO
```

```

-- Schedule Backup Jobs
-- Full Backup: Weekly (Sunday 2 AM)
-- Differential Backup: Daily (2 AM)
-- Transaction Log Backup: Every 6 hours

-- Backup to Azure Blob Storage (Optional)
CREATE CREDENTIAL [AzureStorageCredential]
WITH
    IDENTITY = 'SHARED ACCESS SIGNATURE',
    SECRET = '<SAS_TOKEN>';
GO

BACKUP DATABASE DM_FTI_DW
TO URL =
N'https://[storage_account].blob.core.windows.net/backups/DM_FTI_DW.bak'
WITH
    CREDENTIAL = 'AzureStorageCredential',
    COMPRESSION;
GO

```

5. Step 5: User Acceptance Testing

Tujuan: Validasi sistem dengan end-users

Aktivitas:

5.1. Create Test Cases

Test ID	Scenario	Expected Result	Status	Notes
TC001	View total students per program	Dashboard shows correct count	Pass	
TC002	Filter by academic year	Data filtered correctly	Pass	
TC003	Export report to Excel	File downloads successfully	Pass	
TC004	Drill-down from program to student	Detail view appears	Pass	

TC005	ETL runs successfully	Data updated in warehouse	Pass	

5.2. Conduct UAT Sessions

- Schedule meeting dengan stakeholders
- Demonstrasi dashboard dan features
- Collect feedback dan bug reports
- Document change requests

5.3. Performance Testing

- Test dengan concurrent users
- Measure dashboard load time
- Test ETL with production data volume
- Identify and fix bottlenecks

5.4. Refinement

- Fix bugs dari UAT
- Implement requested features
- Optimize slow queries
- Update documentation

6. Step 6: Documentation

Tujuan: Menyiapkan dokumentasi lengkap untuk operasional

Dokumen yang Diperlukan:

6.1. System Architecture Document

- High-level architecture diagram
- Technology stack
- Data flow diagram
- Deployment architecture

6.2. Data Dictionary (Update dari Misi 1)

- Complete table and column definitions
- Business rules
- Data lineage

6.3. ETL Documentation

- ETL process flow
- Transformation rules
- Error handling procedures
- Schedule and dependencies

6.4. User Manual

- How to access dashboard
- How to navigate reports
- How to apply filters
- How to export data
- FAQ section

6.5. Operations Manual

- Daily operations checklist
- How to monitor ETL jobs
- How to troubleshoot common issues
- Backup and recovery procedures
- Contact information for support

6.6. Security Documentation

- User roles and permissions
- Access request procedures
- Password policies
- Audit trail queries